Neologik Onboarding Script — `Install-NeologikEnvironment.ps1`

Version: v1.9.0 (develop) · **Requires:** PowerShell 7.4+, operator with **Subscription Owner** + **Entra Global Administrator**

An idempotent, checkpoint-resumable PowerShell script that provisions a customer's Azure environment as stage one of onboarding. It authenticates the operator, validates permissions and quota, creates all foundational Azure/Entra resources, then exports a config JSON (`NeologikConfiguration_<timestamp>.json`) consumed by the deployment pipelines. Central design principle: **least-privilege, resource-group-scoped grants** — nothing the pipeline runs afterward needs standing subscription-level rights.

Flow

Runs 16 ordered steps (`Start-NeologikOnboarding`), checkpointing to `NeologikCheckpoint.json` after each. A failure exits `2` (permissions) or `1` (other) and preserves the checkpoint; re-running offers resume. On success the checkpoint is deleted. A pre-flight step also warns (non-fatally) on insufficient vCPU or Azure OpenAI `gpt-5.4-mini` quota in the target region.

Permission validation

Up-front, before any prompts (`Test-RequiredPermissions`):

- **Subscription Owner** — verified via `Get-AzRoleAssignment`. Missing → hard fail.
- **Global Administrator** — verified via Graph directory-role membership. If the *check itself* can't run (e.g. Graph token error), membership is unknown and the operator is prompted to proceed at their own risk. Graph app-role grants additionally need Privileged Role Administrator or Global Administrator.

What it creates

Resource	Notes
Resource group	Pre-created here so the pipeline never needs subscription scope
Security groups	incl. *Neologik Admin User Group* (operator + invited guests)
Guest user invitations	From `NeologikGuestUsers.txt`
App registration + SP	Deployment identity; added to Admin group
Key Vault	`kvneo<org><env><index>`; stores the SP client secret
Storage account + blob container	For certificates

Resource	Notes
Managed identities	Script Runner Service Connection; SQL Managed Identity
Azure resource providers	Registered subscription-wide
Cost Management email	Subscription-scoped scheduled report (run as Owner)
Config export	`NeologikConfiguration_<timestamp>.json` handoff artifact

Azure access it grants

All grants are **resource-group-scoped** unless noted. No standing subscription roles are assigned to the pipeline SP or the admin group.

Principal	Scope	Roles
Deployment SP (app reg)	Resource group	Contributor · Key Vault Certificates Officer · Key Vault Secrets Officer · Storage Blob Data Contributor · AKS RBAC Cluster Admin · Monitoring Contributor · **Role Based Access Control Administrator**
Deployment SP	Entra / Graph	`Application.Read.All` + `Application.ReadWrite.OwnedBy` (replaces tenant-wide *Application Administrator*)
Admin User Group	Resource group	Contributor · AKS Cluster Admin Role · AKS RBAC Cluster Admin · Key Vault Administrator · Storage Blob Data Owner · Storage Table Data Contributor · Cognitive Services Data Reader · Cognitive Services OpenAI Contributor · Search Index Data Reader · Azure AI Developer
Script Runner MI	Resource group	Contributor · Key Vault Secrets Officer · Azure Bot Service Contributor Role
SQL MI	—	none (consumed later)

Notes:

- `Role Based Access Control Administrator` replaces legacy sub-scoped *User Access Administrator* — it grants `roleAssignments/write` so deployment Bicep can create in-template assignments (kubelet Managed Identity Operator, workload-identity, kubelet AcrPull) without deny-assignment powers.
- `Application.ReadWrite.OwnedBy` only manages apps the SP **owns**, so secret-rotation flows must first register the identity as owner.
- If the environment uses a **shared** container-registry RG, the kubelet AcrPull assignment lands there and needs a separate one-time RBAC Administrator grant on that RG.

Key parameters

`OrganizationCode` (1–3 chars) · `EnvironmentType` (`dev` / `prd` ; `dmo` / `test` / `stg` accepted but undocumented) · `AzureRegion` (`uksouth`) · `TenantId` · `SubscriptionName` · `KeyVaultName` · `SkipPowerShellUpdate` · `SkipModuleInstall`